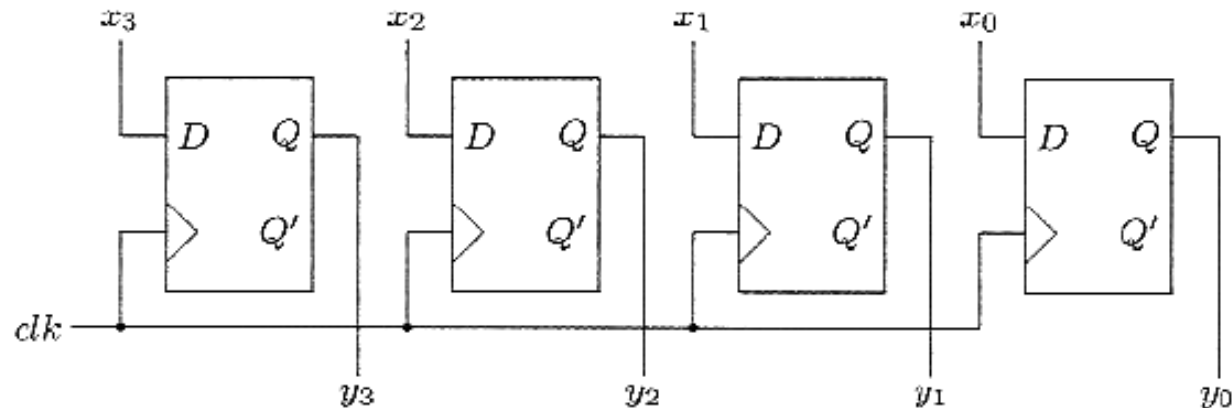

Macchine sequenziali
notevoli:
registri e contatori

Registri

- I registri sono circuiti sequenziali ampiamente utilizzati nei calcolatori per immagazzinare informazioni
- Un registro è composto in generale da un insieme di flip-flop, che memorizzano i dati, e da un insieme di porte logiche che implementano opportune transizioni di stato (determinano il dato che deve essere trasferito nei flip-flop)
- I registri possono essere classificati in base alla modalità con cui i dati sono scritti e letti: in entrambi i casi le modalità possibili sono *in parallelo* o *in serie*

Registro parallelo-parallelo

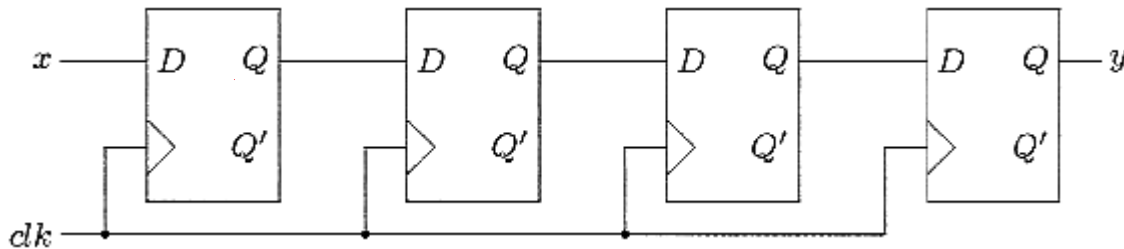
- Il registro parallelo-parallelo è composto solo da flip-flop
- I flip-flop sono sincronizzati sul fronte di un clock (in modo che la lettura da tutti i flip-flop avvenga nello stesso istante)
 - I valori binari presenti sugli ingressi sono memorizzati nei flip-flop
 - Le uscite possono essere esaminate per leggere il contenuto del registro



Registro a scorrimento o shift register (serie-serie)

- I bit immagazzinati attraverso l'ingresso seriale x vengono traslati di una posizione in sincronia con un impulso di clock, mentre in uscita viene presentato un bit alla volta ad ogni impulso

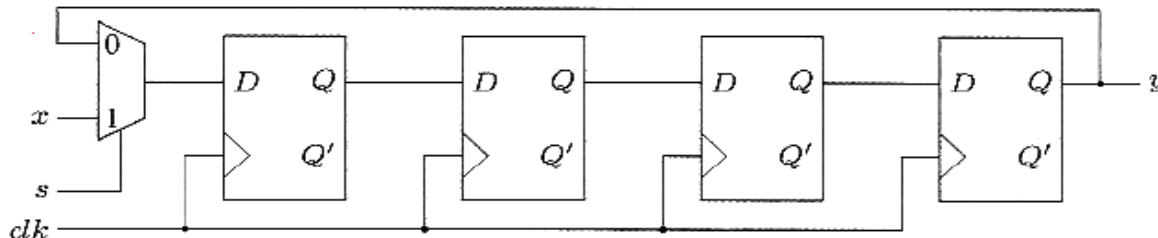
I registri a scorrimento sono formati logicamente da una catena di flip-flop connessi in cascata, in cui l'uscita di ogni flip-flop è connessa all'ingresso del flip-flop successivo



Attenzione: i bistabili non possono essere latch altrimenti ci sarebbe il rischio di far propagare un dato in più bistabili (a limite l'abilitazione dovrebbe essere molto stretta).

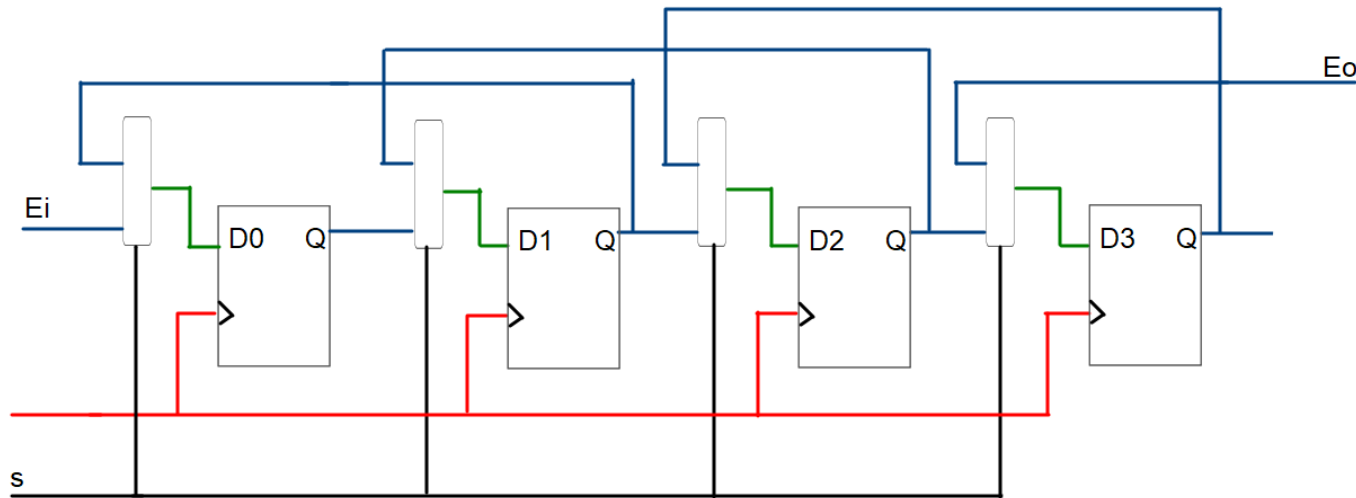
Registro a scorrimento circolare

- Un registro a scorrimento circolare pone in ingresso al primo flip-flop l'uscita dell'ultimo.
- Se volessimo progettare un registro che sa funzionare a scelta o come shift register classico o come shift register circolare, sarebbe possibile inserire un MUX all'ingresso del primo flip-flop, con un segnale di controllo s che permette di decidere se caricare un ingresso seriale x ($s=1$) oppure porre in ingresso il valore in uscita dall'ultimo bistabile
- Si noti che, in un registro a scorrimento circolare, bisognerebbe comunque prevedere una modalità di inserimento dei dati nel registro, anche in parallelo volendo



Registro a scorrimento sinistra/destra

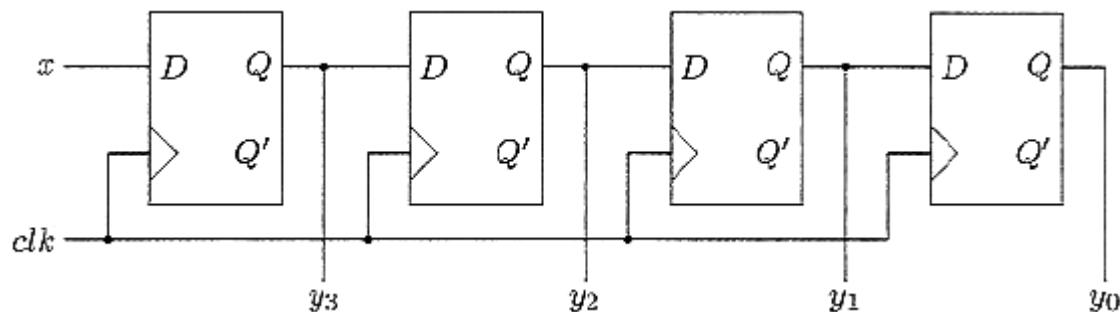
- Se volessimo realizzare un registro a scorrimento in grado di fare shift a destra o a sinistra a comando, basterebbe inserire **un MUX in ingresso ad ogni flip-flop**: ogni MUX ha un ingresso di selezione s , tale che quando $s=0$, si shifta a sinistra, e quando $s=1$ si shifta a destra
- Gli ingressi di ogni MUX devono essere presi quindi (per $s=0$) dall'uscita del flip-flop a destra di quello considerato, oppure (per $s=1$) dall'uscita del flip-flop a sinistra.



Ei è un ingresso
seriale esterno,
Eo è un'uscita
seriale

Registro serie-parallelo

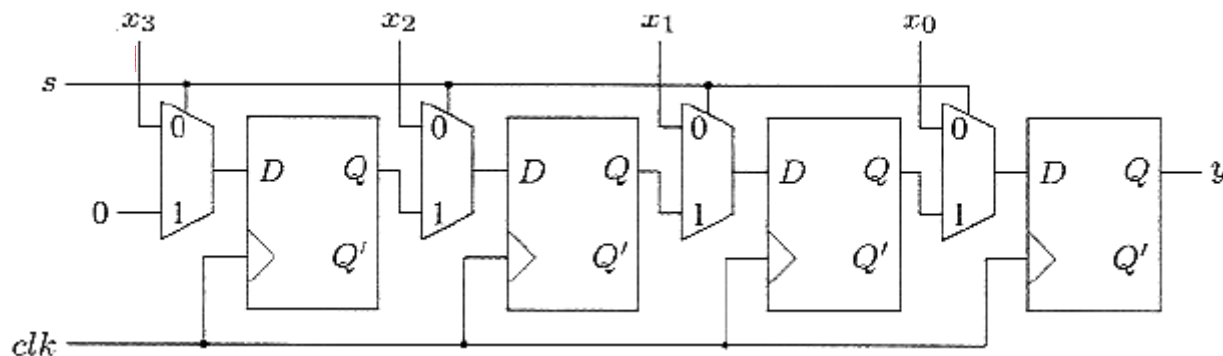
- In questi registri la parola di input viene caricata un bit alla volta grazie ad un solo ingresso x , mentre la lettura del valore memorizzato avviene in parallelo (utile nei sistemi di telecomunicazioni dove bisogna convertire bit ricevuti in serie in byte – *conversione serie-parallelo*)
 - L'architettura deve far scorrere i bit man mano che arrivano per evitare la sovrascrittura, quindi questo registro è costruito a partire da un registro a scorrimento



L'uscita di un bistabile va in ingresso al successivo: ad ogni fronte del clock ogni bistabile campiona il segnale in ingresso e lo mantiene fino al prossimo fronte

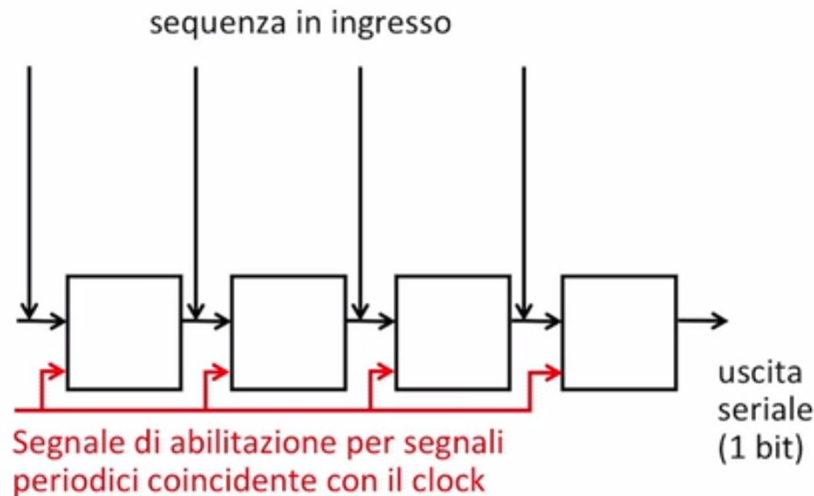
Registri parallelo-serie (a scorrimento con caricamento parallelo)

- Questi registri presentano un ingresso parallelo su k bit e una uscita seriale su un bit: ancora una volta è possibile costruirli partendo da un registro a scorrimento
- Per poter utilizzare la stessa architettura come registro a scorrimento a caricamento seriale o parallelo a scelta, è possibile usare dei *multiplexer* in ingresso a ciascun bistabile per selezionare l'ingresso dato parallelo oppure il segnale che viene dal bistabile precedente (serie)
- I multiplexer sono comandati da un segnale di selezione s (**shift**):
 - quando $s=0$ i bistabili caricano gli ingressi paralleli, quando $s=1$ il registro si comporta da shift register semplice



Registri a scorrimento per la generazione di sequenze

- Il registro a scorrimento con caricamento parallelo è impiegabile nella generazione di sequenze: scrivendo una sequenza in parallelo nel registro ed usando il clock è possibile ritornare in uscita ogni singolo elemento della sequenza, uno alla volta.



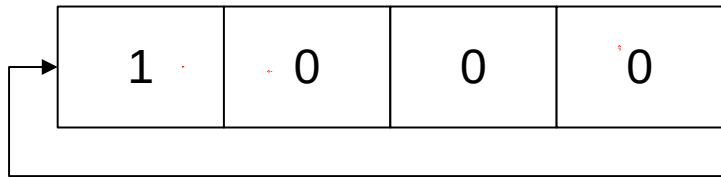
Collegando l'uscita con l'ingresso in modalità circolare è possibile generare sequenze periodiche

Registri a scorrimento per effettuare moltiplicazioni e divisioni per potenze di 2

- Le divisioni o moltiplicazioni per due si possono realizzare mediante operazioni di scorrimento verso destra o sinistra rispettivamente dell'operando memorizzato in un registro di questo tipo.
- Più in generale, uno shift a sinistra di i posizioni ($i > 0$) equivale alla moltiplicazione per 2^i , mentre uno shift a destra equivale alla divisione per 2^i

Registri a scorrimento per realizzare contatori

- Un registro a scorrimento di n bit può essere utilizzato facilmente come contatore mod- n



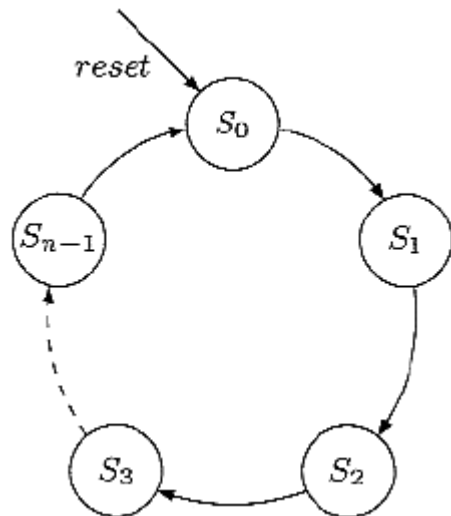
1000	Il conteggio massimo
0100	viene raggiunto ad
0010	esempio quando l'1
0001	arriva nell'ultima
1000	posizione
....	

Questa architettura è più dispendiosa del progetto classico poiché usa 4 flip-flop laddove invece ne basterebbero 2

Contatori

Contatori

- Nella sua forma più semplice un contatore è una macchina di Moore priva di ingressi dato, che evolve attraverso una sequenza predefinita di stati di conteggio in corrispondenza di un impulso di conteggio (quando l'impulso non è presente il contatore non evolve)
 - per i problemi di tempificazione trattati nelle lezioni precedenti, l'impulso di conteggio deve avere una durata molto piccola per non consentire un'evoluzione fra più di due stati consecutivi, oppure se ne deve considerare la variazione da 0 a 1 (fronte di salita) o da 1 a 0 (fronte di discesa) per cambiare stato



La lunghezza n della sequenza di stati attraversati da un contatore si dice *modulo*



Un contatore parte tipicamente da uno stato di *reset*, in cui si riporta ogni volta che in ingresso viene posto un **segnale di reset**

L'uscita di un contatore Z è il **valore di conteggio** corrente del contatore

Progetto di contatori come macchine sincrone

- I contatori possono essere progettati come macchine sincrone seguendo la procedura vista per le macchine sincrone in generale: si parte dall'automa, si scelgono i flip-flop e si procede con la sintesi, identificando la rete combinatoria minima che consente di determinare lo stato prossimo.
 - Nella loro forma più semplice i contatori hanno solo il segnale di tempificazione (impulso di conteggio o clock) in ingresso, ma potrebbero esserci ingressi aggiuntivi, come il segnale di abilitazione (EN), il segnale di RESET e il segnale di LOAD
 - L'uscita del contatore in genere coincide con lo stato prossimo ed è rappresentata semplicemente dalle uscite ordinate dei flip-flop che lo compongono
 - Spesso i contatori hanno un'uscita detta *divisore* o riporto CO, alta quando si raggiunge il valore massimo di conteggio (per un contatore mod-n, l'uscita divisore è alta quando si raggiunge il valore n-1)

Progetto di contatori sincroni – esempio (1/7)

- Si vuole progettare un contatore sincrono **modulo 16** dotato del solo ingresso di conteggio
 - La macchina ha 16 stati codificati su 4 bit Q_3, Q_2, Q_1, Q_0 , e non ha ingressi nè uscite aggiuntivi

Nota: quando il circuito raggiunge lo stato 1111, passerà, nella successiva transizione, allo stato 0000, iniziando un nuovo ciclo di conteggio

Present state				Next state			
Q_3	Q_2	Q_1	Q_0	Q_3	Q_2	Q_1	Q_0
0	0	0	0	0	0	0	1
0	0	0	1	0	0	1	0
0	0	1	0	0	0	1	1
0	0	1	1	0	1	0	0
0	1	0	0	0	1	0	1
0	1	0	1	0	1	1	0
0	1	1	0	0	1	1	1
0	1	1	1	1	0	0	0
1	0	0	0	1	0	0	1
1	0	0	1	1	0	1	0
1	0	1	0	1	0	1	1
1	0	1	1	1	1	0	0
1	1	0	0	1	1	0	1
1	1	0	1	1	1	1	0
1	1	1	0	1	1	1	1
1	1	1	1	0	0	0	0

Progetto di contatori sincroni – esempio (2/7)

Posizionamento dei flip-flop:

Esempio con flip-flop di tipo JK edge-triggered

Present state				Next state				Flip-flop inputs							
Q_3	Q_2	Q_1	Q_0	Q_3	Q_2	Q_1	Q_0	J_{Q3}	K_{Q3}	J_{Q2}	K_{Q2}	J_{Q1}	K_{Q1}	J_{Q0}	K_{Q0}
0	0	0	0	0	0	0	1	0	X	0	X	0	X	1	X
0	0	0	1	0	0	1	0	0	X	0	X	1	X	X	1
0	0	1	0	0	0	1	1	0	X	0	X	X	0	1	X
0	0	1	1	0	1	0	0	0	X	1	X	X	1	X	1
0	1	0	0	0	1	0	1	0	X	X	0	0	X	1	X
0	1	0	1	0	1	1	0	0	X	X	0	1	X	X	1
0	1	1	0	0	1	1	1	0	X	X	0	X	0	1	X
0	1	1	1	1	0	0	0	1	X	X	1	X	1	X	1
1	0	0	0	1	0	0	1	X	0	0	X	0	X	1	X
1	0	0	1	1	0	1	0	X	0	0	X	1	X	X	1
1	0	1	0	1	0	1	1	X	0	0	X	X	0	1	X
1	0	1	1	1	1	0	0	X	0	1	X	X	1	X	1
1	1	0	0	1	1	0	1	X	0	X	0	0	X	1	X
1	1	0	1	1	1	1	0	X	0	X	0	1	X	X	1
1	1	1	0	1	1	1	1	X	0	X	0	X	0	1	X
1	1	1	1	0	0	0	0	X	1	X	1	X	1	X	1

Flip-Flop Excitation Tables			
(a) JK Flip-Flop			
$Q(t)$	$Q(t+1)$	J	K
0	0	0	X
0	1	1	X
1	0	X	1
1	1	X	0

Si vede facilmente
che:

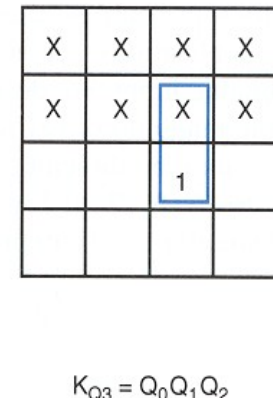
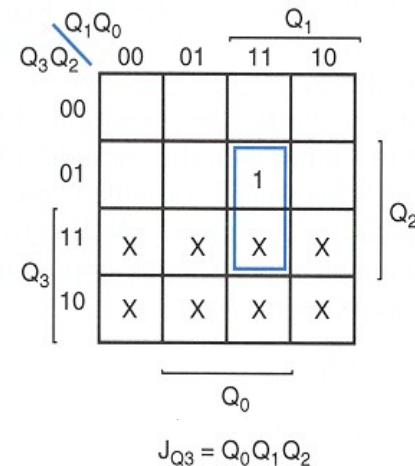
$$J_{Q0} = K_{Q0} = 1$$

Progetto di contatori sincroni – esempio (3/7)

Ottimizzazione delle espressioni dei segnali di posizionamento

- JQ3 e KQ3

Present state				Next state					
Q ₃	Q ₂	Q ₁	Q ₀	Q ₃	Q ₂	Q ₁	Q ₀	J _{Q3}	K _{Q3}
0	0	0	0	0	0	0	1	0	X
0	0	0	1	0	0	1	0	0	X
0	0	1	0	0	0	1	1	0	X
0	0	1	1	0	1	0	0	0	X
0	1	0	0	0	1	0	1	0	X
0	1	0	1	0	1	1	0	0	X
0	1	1	0	0	1	1	1	0	X
0	1	1	1	1	0	0	0	1	X
1	0	0	0	1	0	0	1	X	0
1	0	0	1	1	0	1	0	X	0
1	0	1	0	1	0	1	1	X	0
1	0	1	1	1	1	0	0	X	0
1	1	0	0	1	1	0	1	X	0
1	1	0	1	1	1	1	0	X	0
1	1	1	0	1	1	1	1	X	0
1	1	1	1	0	0	0	0	X	1



Progetto di contatori sincroni – esempio (4/7)

Ottimizzazione delle espressioni dei segnali di posizionamento

- JQ2 e KQ2

Present state				Next state				Flip-flo	
Q ₃	Q ₂	Q ₁	Q ₀	Q ₃	Q ₂	Q ₁	Q ₀	J _{Q2}	K _{Q2}
0	0	0	0	0	0	0	1	0	×
0	0	0	1	0	0	1	0	0	×
0	0	1	0	0	0	1	1	0	×
0	0	1	1	0	1	0	0	1	×
0	1	0	0	0	1	0	1	×	0
0	1	0	1	0	1	1	0	×	0
0	1	1	0	0	1	1	1	×	0
0	1	1	1	1	0	0	0	×	1
1	0	0	0	1	0	0	1	0	×
1	0	0	1	1	0	1	0	0	×
1	0	1	0	1	0	1	1	0	×
1	0	1	1	1	1	0	0	1	×
1	1	0	0	1	1	0	1	×	0
1	1	0	1	1	1	1	0	×	0
1	1	1	0	1	1	1	1	×	0
1	1	1	1	0	0	0	0	×	1

		1	
X	X	X	X
X	X	X	X
		1	

$$J_{Q2} = Q_0 Q_1$$

X	X	X	X
		1	
		1	
X	X	X	X

$$K_{Q2} = Q_0 Q_1$$

Progetto di contatori sincroni – esempio (5/7)

Ottimizzazione delle espressioni dei segnali di posizionamento

- JQ1 e KQ1

Present state				Next state				p inputs	
Q ₃	Q ₂	Q ₁	Q ₀	Q ₃	Q ₂	Q ₁	Q ₀	J _{Q1}	K _{Q1}
0	0	0	0	0	0	0	1	0	X
0	0	0	1	0	0	1	0	1	X
0	0	1	0	0	0	1	1	X	0
0	0	1	1	0	1	0	0	X	1
0	1	0	0	0	1	0	1	0	X
0	1	0	1	0	1	1	0	1	X
0	1	1	0	0	1	1	1	X	0
0	1	1	1	1	0	0	0	X	1
1	0	0	0	1	0	0	1	0	X
1	0	0	1	1	0	1	0	1	X
1	0	1	0	1	0	1	1	X	0
1	0	1	1	1	1	0	0	X	1
1	1	0	0	1	1	0	1	0	X
1	1	0	1	1	1	1	0	1	X
1	1	1	0	1	1	1	1	X	0
1	1	1	1	0	0	0	0	X	1

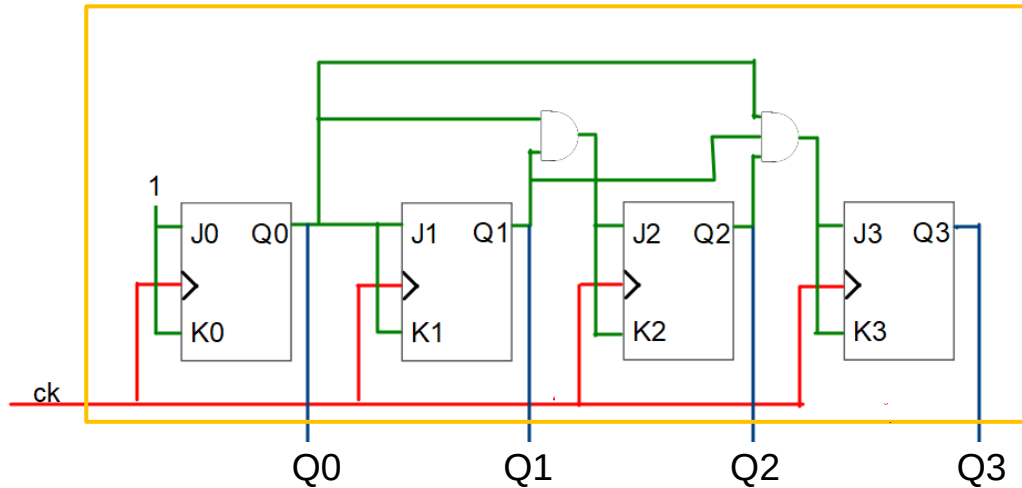
	1	X	X
	1	X	X
	1	X	X
	1	X	X

$$J_{Q1} = Q_0$$

X	X	1	
X	X	1	
X	X	1	
X	X	1	

$$K_{Q1} = Q_0$$

Progetto di contatori sincroni – esempio (6/7)



$$J_0 = K_0 = 1$$

$$J_1 = K_1 = Q_0$$

$$J_2 = K_2 = Q_0 Q_1$$

$$J_3 = K_3 = Q_0 Q_1 Q_2$$

Il flip-flop cui corrisponde il bit meno significativo commuta ad ogni fronte attivo del clock

Ciascuno dei flip-flop successivi commuta in corrispondenza del fronte di attivo del clock, solo quando tutti i bit delle posizioni precedenti meno significative sono uguali a 1

L'uscita del contatore, corrispondente al valore di conteggio, si legge sulle uscite di ciascun flip-flop, con Q3 cifra più significativa e Q0 cifra meno significativa.

Il **segnale di abilitazione EN** potrebbe essere facilmente aggiunto modificando le espressioni dei segnali di posizionamento in questo modo:

$$J_0 = K_0 = 1 \cdot EN$$

$$J_1 = K_1 = Q_0 \cdot EN$$

$$J_2 = K_2 = Q_0 \cdot Q_1 \cdot EN$$

$$J_3 = K_3 = Q_0 \cdot Q_1 \cdot Q_2 \cdot EN$$

L'**uscita CO** è alta quando le uscite dei flip-flop sono tutte alte e EN=1:

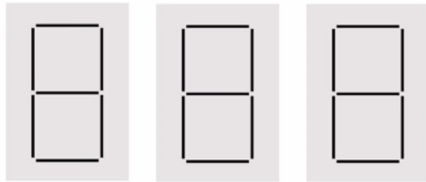
$$CO = Q_0 \cdot Q_1 \cdot Q_2 \cdot Q_3 \cdot EN$$

Progetto strutturale di contatori

- Come abbiamo visto per le macchine combinatorie, anche per le macchine sequenziali è possibile procedere al **progetto strutturale**, sfruttando cioè un insieme di altre macchine notevoli con funzionamento ben definito opportunamente interconnesse
- Nel caso dei contatori, è possibile ottenere un contatore modulo- N utilizzando un approccio strutturale secondo due modalità:
 - **per composizione**, collegando opportunamente k contatori, ciascuno di modulo- M_k , con $M_k < N$
 - **per riduzione**, utilizzando un contatore di modulo $M > N$ e modificando il suo ciclo di conteggio
- In entrambi i casi, è tipicamente necessario prevedere della logica combinatoria aggiuntiva da applicare alle uscite di ciascun contatore utilizzato per realizzare il comportamento desiderato.

Progetto di contatori per composizione (esempio)

- Un contatore può essere realizzato combinando opportunamente contatori di modulo minore.
 - Ad esempio, un contatore modulo 1000 (che quindi conta da 000 a 999) può essere realizzato combinando diversi contatori: uno per le unità, uno per le decine e uno per le centinaia. Ciascuno di essi è un contatore modulo 10 (cioè conta da 0 a 9).



contatore di unità

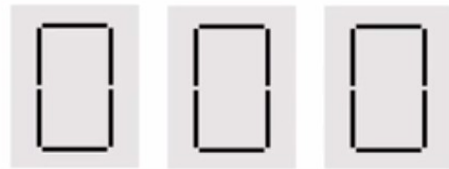
contatore di decine

contatore di centinaia

- E' immediato realizzare che il contatore delle decine incrementa di una unità il suo conteggio solo quando il contatore delle unità passa dal valore 9 al valore 0
- Similmente, il contatore delle centinaia incrementa di una unità il suo conteggio quando sia il contatore delle decine che quello delle unità passano da 9 a 0

Progetto di contatori per composizione (esempio in binario)

- Nel dominio binario, il contatore fondamentale è il **contatore modulo-2**.
- A partire da contatori mod-2, si possono ottenere contatori mod- 2^m semplicemente utilizzando m contatori mod-2 opportunamente interconnessi.
 - Ad esempio, per realizzare un contatore modulo 8 (2^3), si possono utilizzare tre contatori modulo 2



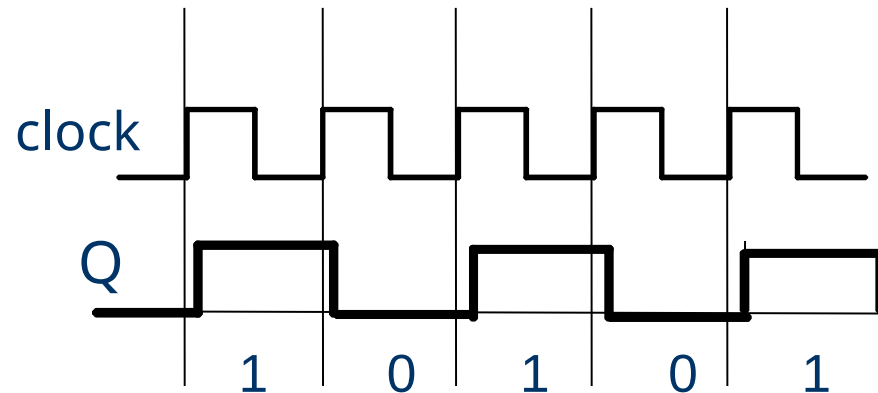
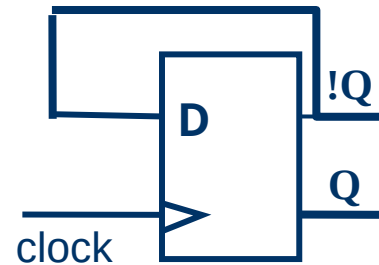
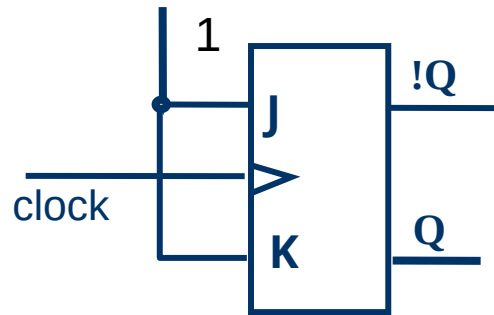
contatore mod-2
contatore mod-2
contatore mod-2

	I=0	I=1	U
q_0	q_0	q_1	0 (000)
q_1	q_1	q_2	1 (001)
q_2	q_2	q_3	2 (010)
q_3	q_3	q_0	3 (011)
q_4	q_4	q_5	4 (100)
q_5	q_5	q_6	5 (101)
q_6	q_6	q_7	6 (110)
q_7	q_7	q_0	7 (111)

- Si osserva che la cifra meno significativa del valore di conteggio (coincidente con lo stato) commuta per ogni impulso di conteggio, la seconda ogni 2 impulsi e la terza ogni 4 impulsi
- Ad ogni cifra è quindi associato un contatore mod-2, ma i 3 contatori contano con una frequenza diversa!

Contatore fondamentale: contatore mod-2

- Il **contatore mod-2** può essere realizzato utilizzando un flip-flop T o un JK in cui si ponga $J=K$, oppure un flip-flop di tipo D retroazionato

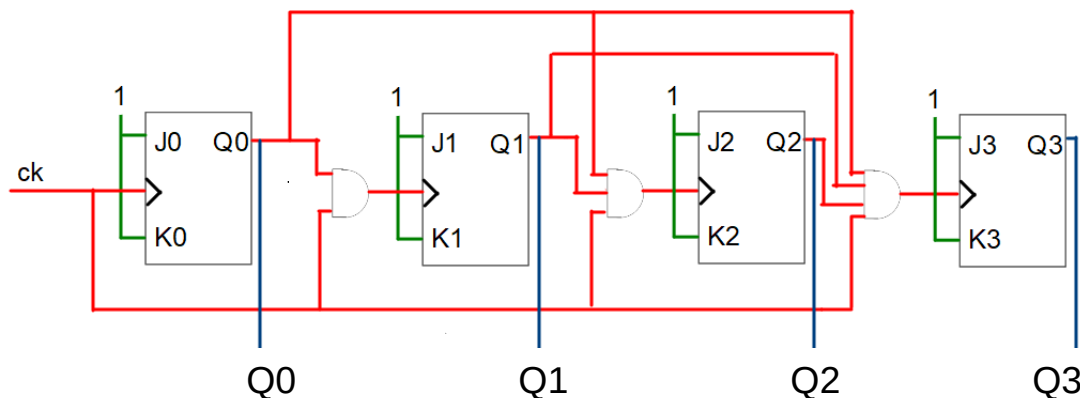


Progetto di contatori per composizione: schemi di collegamento

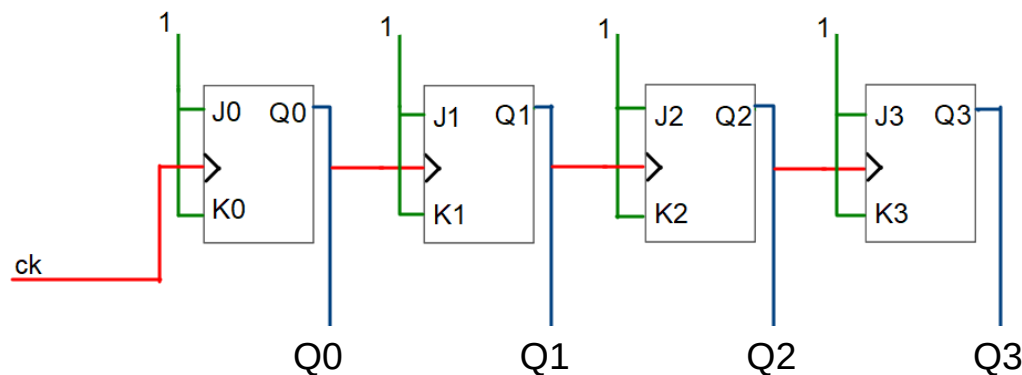
- Quando si compongono diversi contatori per realizzarne uno di modulo complessivamente più grande, è possibile adottare due diverse strategie di collegamento:
 - **schema parallelo:** Il segnale di conteggio è fornito simultaneamente a tutti i contatori (flip-flop) interconnessi, e la frequenza diversa di conteggio è assicurata da una opportuna combinazione del segnale di conteggio e delle uscite dei contatori (flip-flop) precedenti
 - **schema seriale (o a cascata):** il segnale di conteggio è fornito solo al contatore di peso meno significativo, mentre per tutti gli altri è fornito dalle uscite dei contatori precedenti
 - In particolare, ogni contatore (dal secondo in poi) riceve un impulso di conteggio quando quello precedente raggiunge il suo valore massimo di conteggio

Contatori paralleli vs seriali – esempio con flip-flop JK

Esempio: costruire un contatore mod-16 a partire da contatori mod-2 in modalità parallela e seriale



SCHEMA PARALLELO



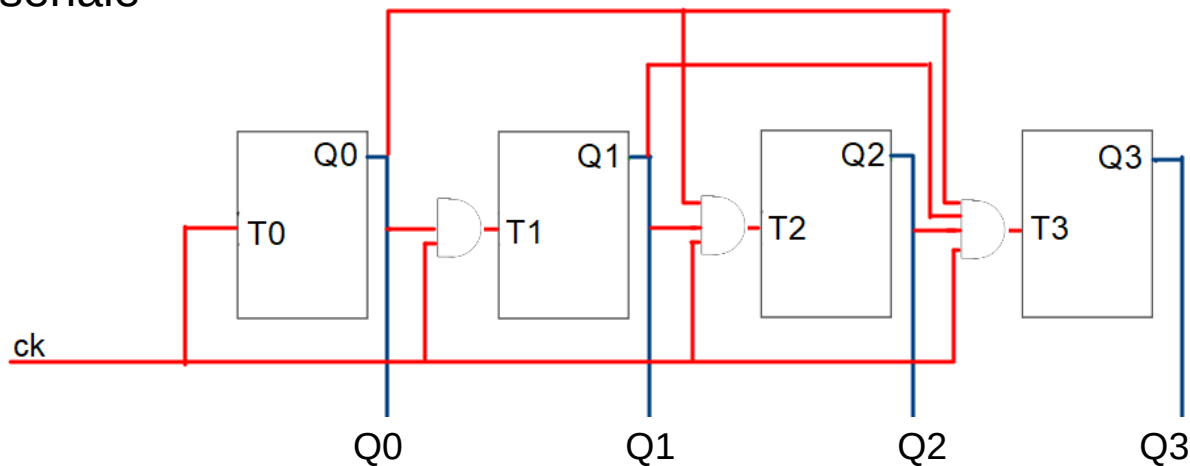
SCHEMA SERIALE

I contatori mod-2 sono realizzati semplicemente da flip-flop JK che funzionano in modalità toggle (in cui sia posto $JK=11$).
I diversi flip-flop commutano con frequenze diverse: il primo ad ogni impulso, il secondo ogni due impulsi, il terzo ogni 4 e il quarto ogni 8

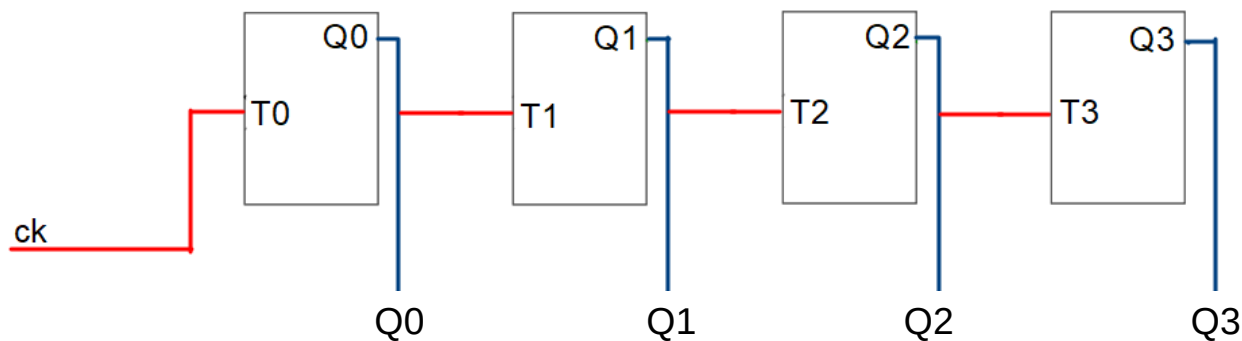
Nella configurazione **parallela**, il clock viene dato a tutti i flip-flop a viene opportunamente manipolato per garantire la diversa frequenza di commutazione;
Nella configurazione **serie** il segnale di conteggio viene dato solo al primo flip-flop, e ogni flip-flop a partire dal secondo prende l'impulso di conteggio dall'uscita del precedente

Contatori paralleli vs seriali – esempio con flip-flop T

Esempio: costruire un contatore mod-16 a partire da contatori mod-2 in modalità parallela e seriale



SCHEMA PARALLELO



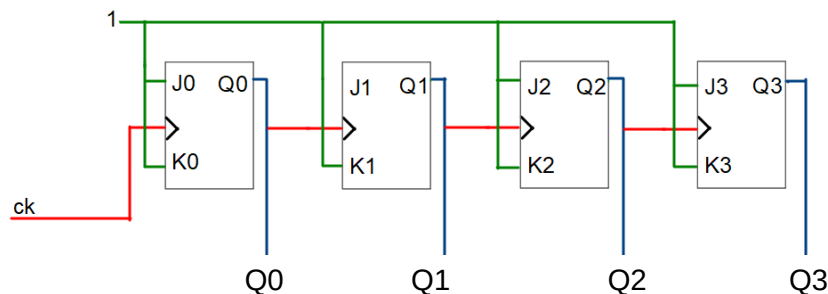
SCHEMA SERIALE

Contatori seriali – ritardi di propagazione

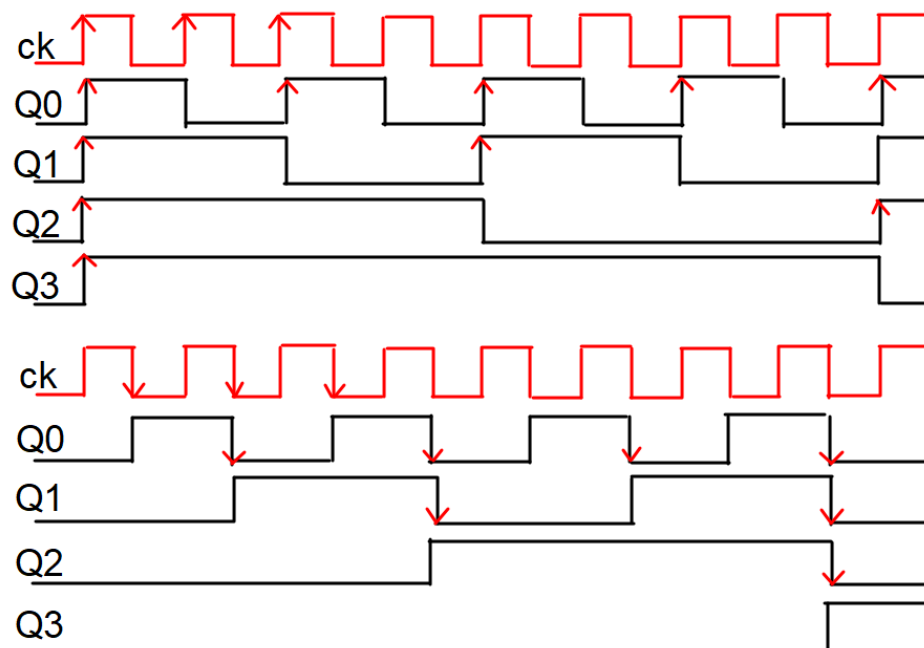
- Lo schema seriale implica una **propagazione** degli impulsi di conteggio lungo la catena di contatori, e quindi per effetto dei ritardi ci saranno degli istanti in cui il valore complessivo di conteggio non è corretto
- Esempio: se un contatore mod-8 deve passare dal valore 3(011) al valore 4(100) ci vorrà del tempo perché l'impulso di conteggio si propaghi dal primo contatore al secondo e dal secondo al terzo: si passerà quindi dal valore 3(01**1**) al valore 2(0**1**0), poi al valore 0(**0**00), e infine al valore 4(**1**00).

Composizione di contatori mod-2: tempificazione

- Nelle configurazioni viste (sia parallela che seriale), è necessario utilizzare **flip-flop di tipo edge-triggered sul fronte di discesa**
 - Devono essere edge-triggered perché devono commutare solo in istanti ben precisi, e devono essere sul fronte di discesa perché altrimenti appena commuta il primo commuterebbe anche il secondo e così via (e avremmo un conteggio a decrescere)

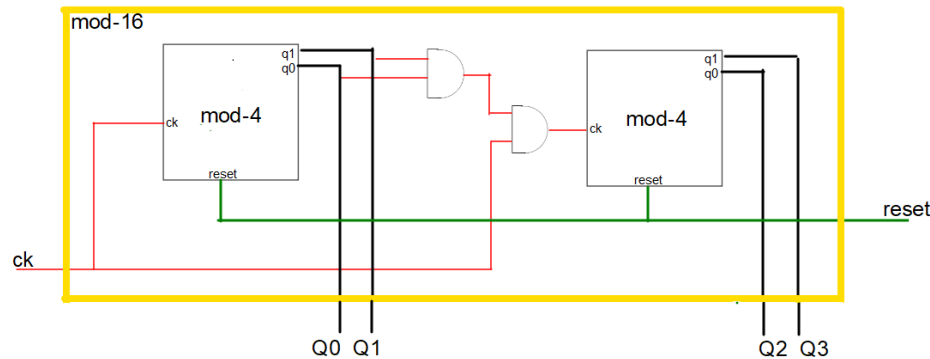


Nota: in realtà il flip-flop meno significativo potrebbe commutare anche sul fronte di salita (mentre tutti gli altri devono commutare solo sul fronte di discesa), ma si preferisce una tempificazione uniforme!



Composizione di contatori generici

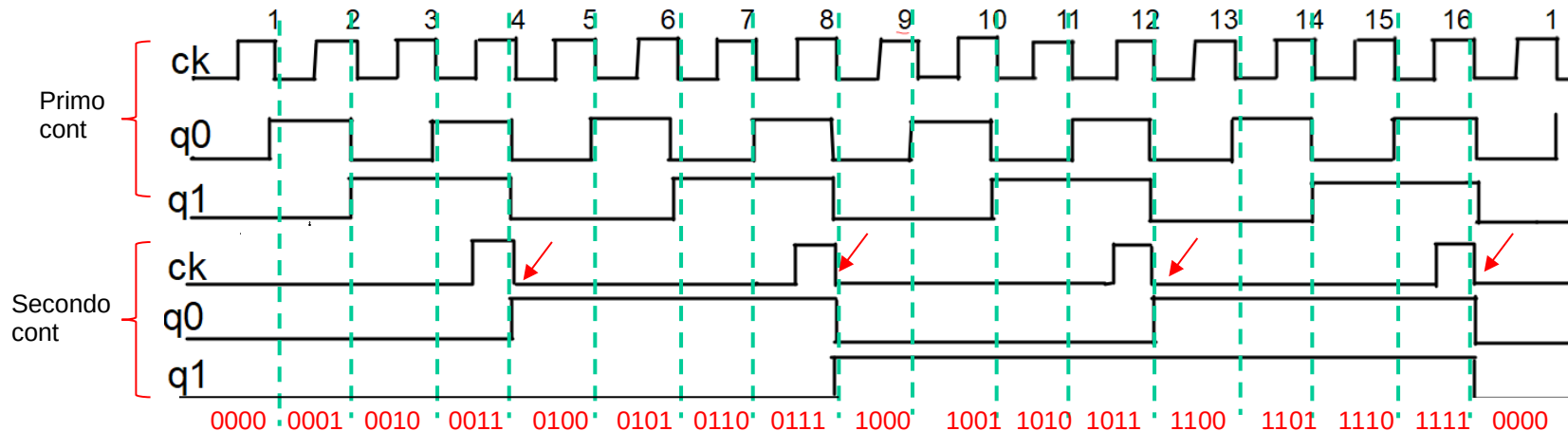
- E' possibile comporre anche contatori di modulo maggiore di 2, e valgono le stesse considerazioni fatte nel caso più semplice



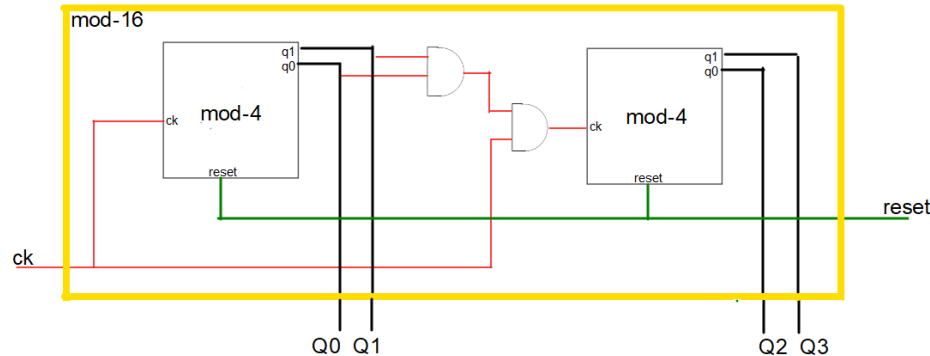
Lo schema a sinistra rappresenta il progetto di un contatore mod-16 a partire da 2 contatori mod-4: come mostrato, il primo contatore (da sinistra) fornisce l'impulso di conteggio al secondo contatore quando raggiunge il valore di conteggio massimo (ossia $q1q0=11$) oppure quando si alza l'uscita divisore, se presente.

L'impulso di conteggio, in particolare, si ottiene ponendo in AND la condizione $q1q0=11$ con l'impulso di conteggio originario ck .

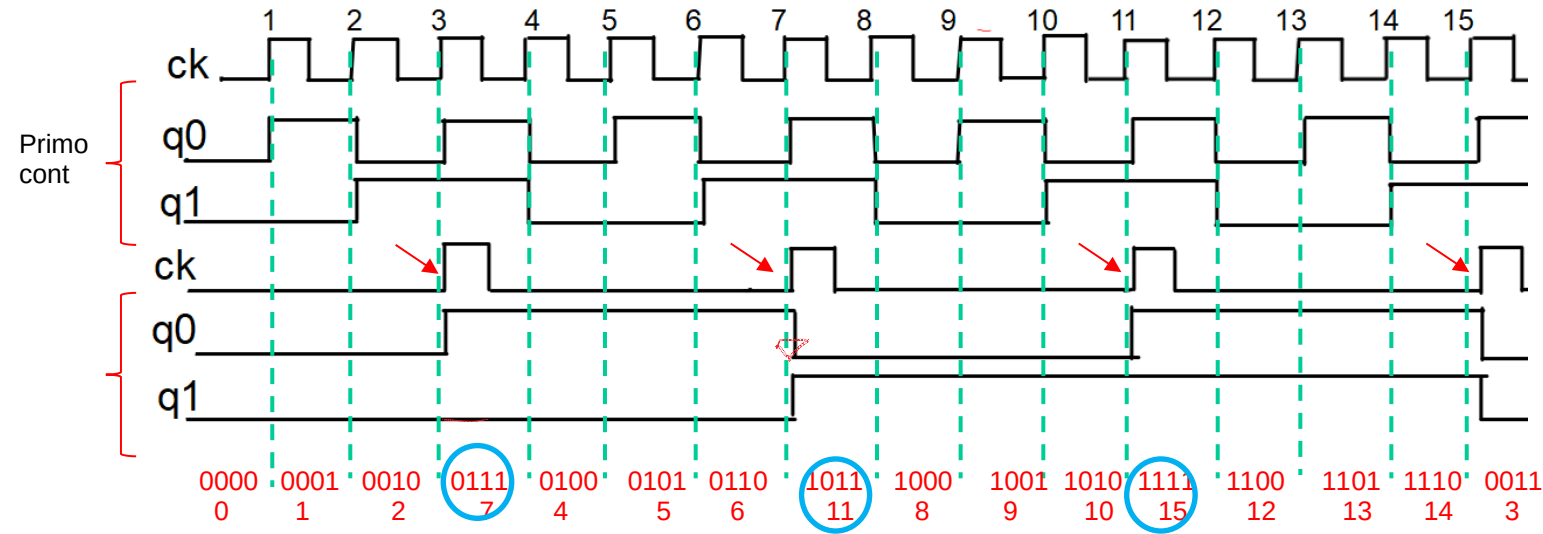
Attenzione: i contatori devono per forza lavorare sul **fronte di discesa** del rispettivo impulso di conteggio!!



Composizione di contatori generici: tempificazione



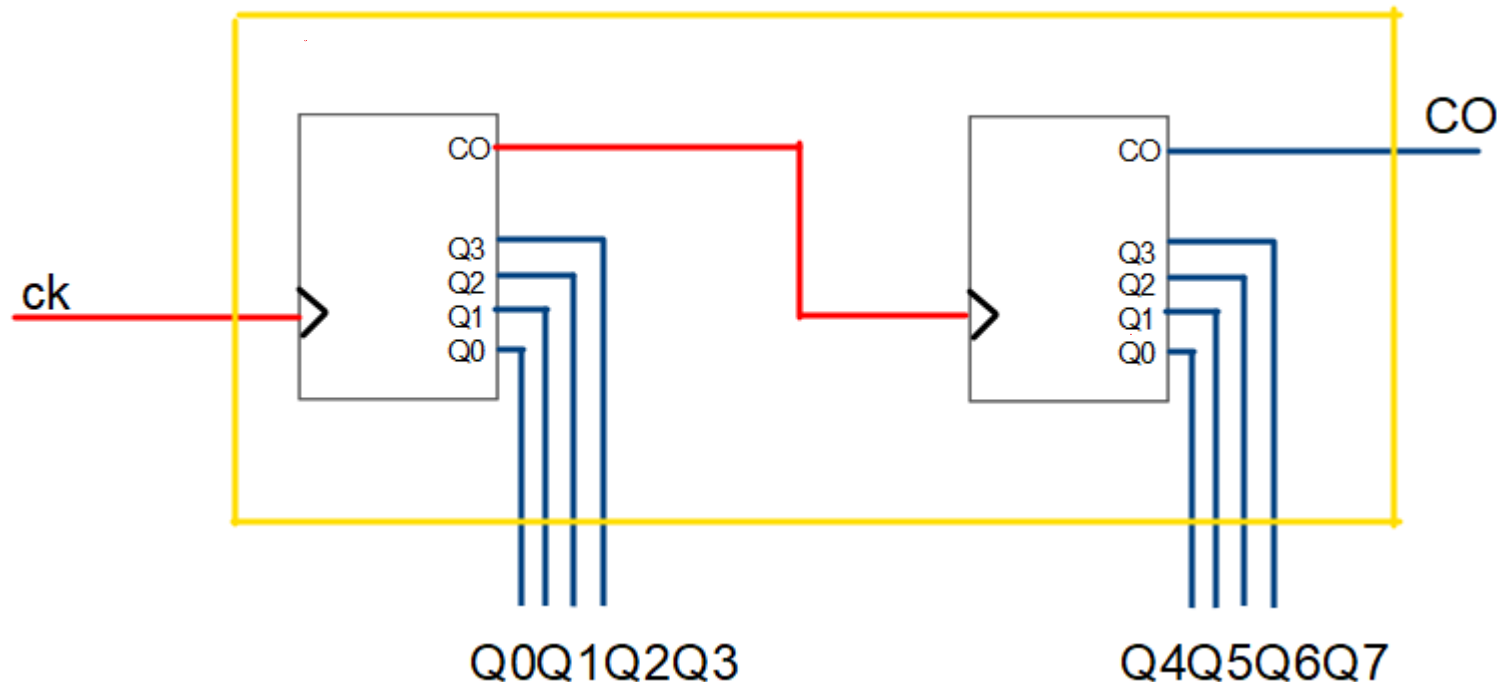
Se il secondo contatore contasse sul fronte di salita del proprio impulso di conteggio il valore di conteggio sarebbe sbagliato!



Nota: in questo esempio si stanno guardando i 2 contatori mod-4 come delle scatole nere, ciascuna in grado di incrementare il valore di conteggio sul fronte di salita del proprio impulso di conteggio. E' chiaro che, essendo composte internamente da 2 flip-flop in cascata, il secondo flip-flop interno a ciascun contatore mod-4 deve commutare sul fronte di discesa dell'impulso, altrimenti il conteggio non sarebbe possibile!

Composizione di contatori generici: esempio

- Es: Realizzare un contatore modulo-256 a partire da contatori modulo-16 dotati di uscita divisore CO
 - È possibile collegare i due contatori in serie sfruttando il fronte di discesa dell'uscita CO del primo



Contatori seriali: valutazioni

I contatori seriali:

- ♦ sono di semplice realizzazione
- ♦ vengono attivate solo le parti del circuito interessate alle transizioni
 - adatti a circuiti che richiedono basso consumo di potenza

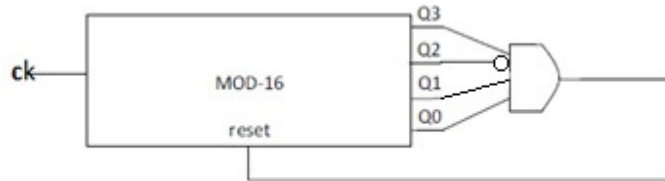
Ma...

- ♦ particolarmente lenti per un alto numero di bit
- ♦ più critici da dimensionare a causa della dipendenza dei ritardi

Progetto di contatori per riduzione

- Il progetto di un contatore mod-N per riduzione (anche detto **in modulo diminuito**) può essere effettuato utilizzando un contatore mod-M con $M=2^n > N$ sfruttando il segnale di **reset** o, laddove presente, il segnale di **load**, che consente di precaricare un valore specifico.
- Sia il segnale di reset che quello di load possono essere **sincroni** o **asincroni**:
 - Un segnale è detto *sincrono* quando il suo valore ha effetto sullo stato di un sistema solo in corrispondenza del fronte attivo di un segnale di sincronizzazione/abilitazione (segnale di conteggio nel caso dei contatori)
 - Un segnale è detto *asincrono* quando una variazione dello stato del sistema dipende dalla variazione del suo valore (indipendentemente dal segnale di sincronizzazione/abilitazione)

Progetto di contatori per riduzione: reset e load sincrono/asincrono (esempio)



In figura è mostrata l'applicazione di una rete combinatoria al contatore mod-16 che abilita il reset in corrispondenza del valore di conteggio 11

Nota: quando il reset/load è sincrono, è possibile ottimizzare la rete combinatoria utilizzata per pilotarlo, poiché tutti i valori di conteggio che seguono quello che ha attivato il segnale costituiscono dei punti di non specificazione per il circuito

Es: si progetti un contatore mod-12 a partire da un modulo-16 utilizzando un segnale di reset sincrono e asincrono

Avendo a disposizione un reset sincrono 1-attivo, tale segnale deve essere posto a 1 quando l'uscita assume valore 11 (1011). In questo modo, al prossimo fronte del segnale di conteggio, il contatore vedrà il reset attivo e si resetterà, e l'uscita assumerà valore 0000 facendo abbassare anche il segnale di reset.

Con un reset asincrono, assumendo che il circuito sia sensibile alla variazione 1->0 di tale segnale, è ancora necessario alzare il reset quando si giunge al valore 11. Sul fronte successivo del clock, poiché il conteggio diventa 12, la condizione per cui reset=1 viene meno, e quindi il reset si abbassa. In corrispondenza di tale variazione il circuito si resetta, e il contatore fornisce 0000 in uscita. In questo caso quindi, per un breve istante, il contatore conterà 12 prima di resettarsi.

Progetto di contatori per riduzione: reset e load sincrono/asincrono (esempio)

- Esempio: realizzare un contatore modulo-10 a partire da uno modulo-16 utilizzando un segnale di load sincrono

Load

Q3Q2 \ Q1Q0	00	01	11	10
	00	01	11	10
00				
01				
11	-	-	-	-
10		1	-	-

$$\text{Load} = Q_3 Q_0$$

Visto che è sincrono, il segnale va attivato quando il conteggio arriva al valore 9 (1001), mentre i valori superiori non sono di interesse (don't care)

